

5. Arquitectura Base y Seguridad: Multi-Tenancy y RBAC

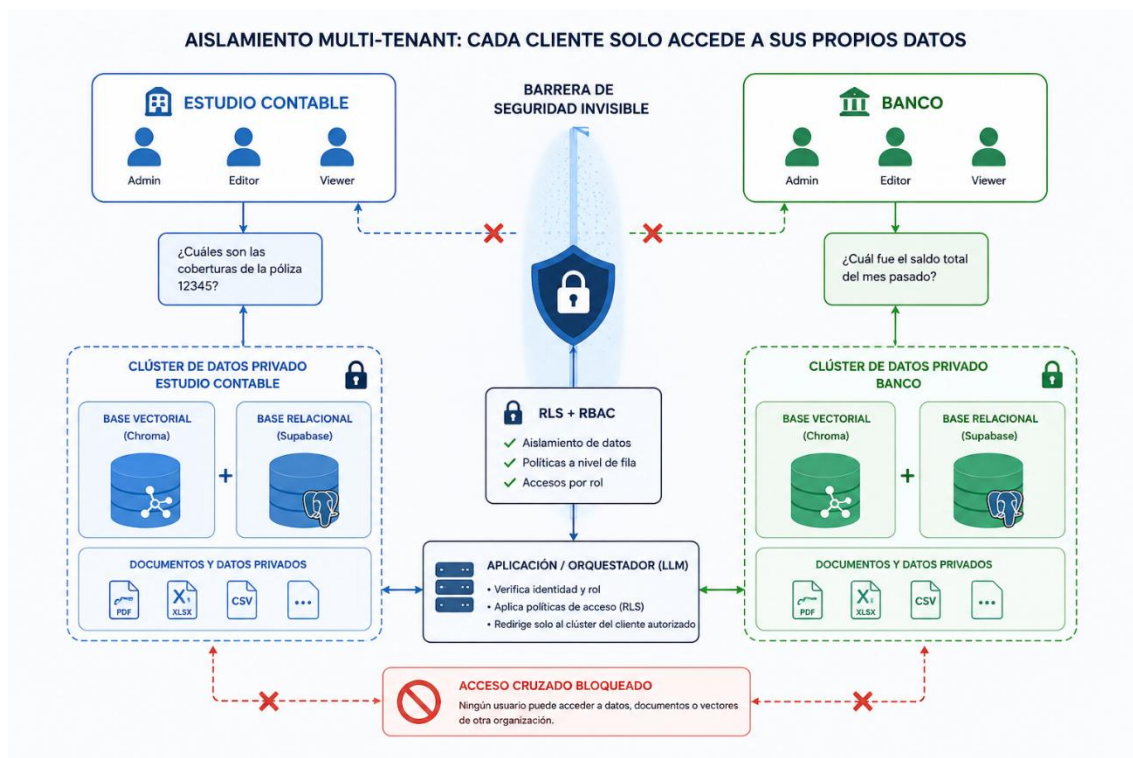
Para asegurar que esta plataforma sea un producto B2B verdaderamente viable y seguro en el mundo real, tuvimos que construir un entorno donde cada empresa opera

- en un espacio totalmente hermético
- garantizando así la privacidad de sus datos
- un funcionamiento constante sin costes desorbitados de servidores.

A continuación, veremos las dos claves de este logro: cómo protegemos la información corporativa y cómo estructuramos nuestra red de forma inteligente.

H2 Seguridad y Aislamiento de la Información Corporativa

El primer pilar es la seguridad. Un cliente jamás debe tener acceso a los documentos o datos de otro cliente. **A esto lo llamamos un entorno aislado.** Además de proteger los datos entre diferentes empresas, **nuestro sistema controla estrictamente quién entra y qué puede hacer cada empleado dentro de su propia organización.** Si un empleado no tiene los permisos suficientes, la plataforma simplemente se adapta y oculta botones u opciones críticas, asegurando que un usuario básico pueda consultar la Inteligencia Artificial, pero nunca borrar información estratégica.



H3 **Gestión de Identidad en el Edge (Clerk):**

Hemos integrado Clerk como solución de autenticación moderna compatible con arquitecturas Edge, evaluando el acceso en el middleware de Next.js antes de renderizar la página y ofreciendo flujos de acceso avanzados (Email y credenciales clásicas).

H3 **Control de Acceso Basado en Roles (RBAC) y Sincronización:**

Clerk emite un Token JWT con un `publicMetadata` que contiene el `orgId` (identificador de la empresa) y el `role` (`admin`, `editor`, `viewer`). Un hook especializado en React

(`AuthContext.tsx`) captura este JWT en el inicio de sesión y sincroniza el perfil mediante una operación `upsert` en la tabla relacional `user\_profiles` de Supabase.

H3 Aislamiento Físico de Datos (Row Level Security - RLS)

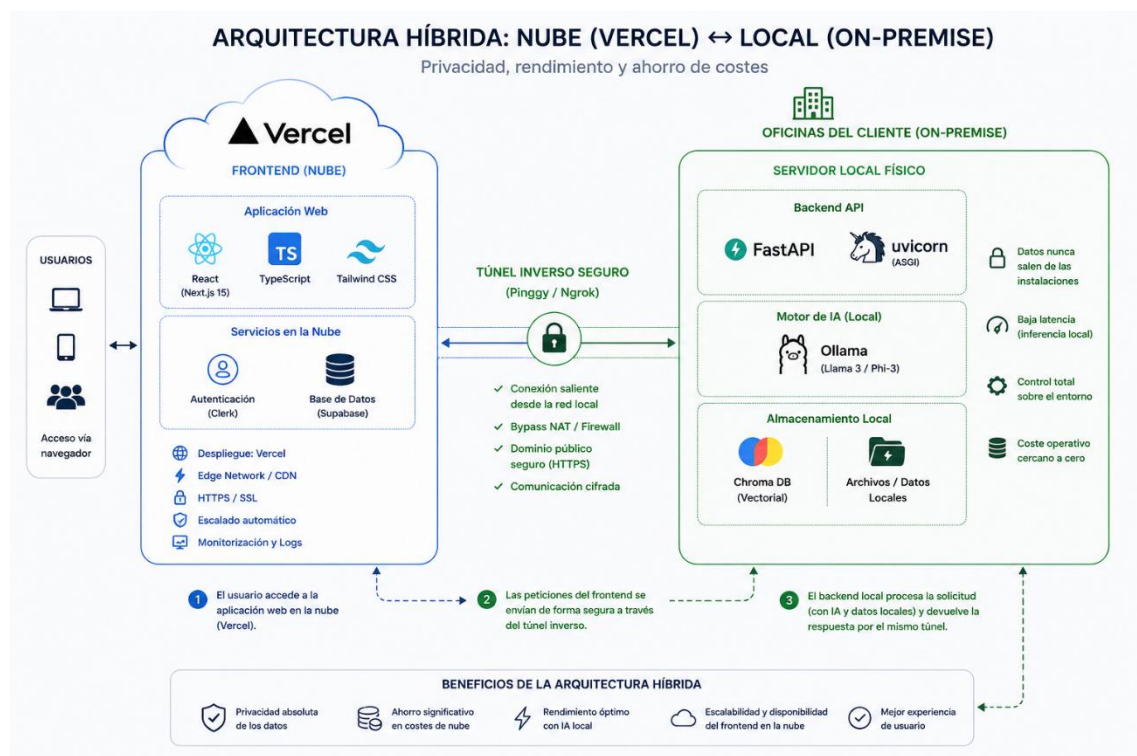
Para garantizar el Multi-Tenancy a nivel de infraestructura profunda, implementamos políticas de seguridad a nivel de fila (RLS) nativas de PostgreSQL en Supabase. Al inyectar dinámicamente el `orgId` en cada consulta, la base de datos rechaza a nivel criptográfico cualquier lectura o escritura de registros que no pertenezcan a la organización del usuario, neutralizando el riesgo de "Cross-Tenant Data Leakage".

H3 **Prevención del Cross-Tenant Data Leakage en la Capa Visual:**

La arquitectura de roles condiciona completamente el DOM de Next.js. A los perfiles con rol `viewer`, el servidor les re-renderiza la interfaz bloqueando dinámicamente el acceso a funcionalidades de manipulación de archivos y diccionarios corporativos, creando una degradación elegante del sistema.

H2 Despliegue Híbrido: Privacidad Absoluta y Ahorro de Costes

El segundo gran reto operativo fue la conectividad. Alquilar potentes ordenadores en la nube para alojar y hacer funcionar nuestro cerebro de Inteligencia Artificial hubiera disparado los costes a cientos de euros mensuales, haciendo el proyecto inviable. Por ello, diseñamos una arquitectura "híbrida": la interfaz gráfica y la aplicación web que usan los clientes están alojadas en la nube, pero todo el núcleo de razonamiento de la Inteligencia Artificial se ejecuta directamente en los servidores locales físicos de la empresa. Gracias a esto, no pagamos por servidores en la nube, las respuestas son instantáneas y la empresa mantiene el control 100% sobre sus propios datos sin depender de terceros.



H3 ****Túnel Reverso para Bypassing NAT:****

Al ejecutar el frontend en Vercel (Cloud) y el backend en FastAPI local (On-Premise), la red Serverless de Vercel arrojaba errores 503/404 por culpa de los cortafuegos y el NAT local. Implementamos un túnel reverso persistente y seguro mediante ****Pinggy / Ngrok****, inyectando un dominio estático (``PYTHON_BACKEND_URL``) en las variables de entorno de Vercel.

H3 ****Resolución del Hydration Mismatch en Next.js:****

A nivel de interfaz, el nuevo ***App Router*** de Next.js generó conflictos de hidratación porque el servidor (Vercel) pre-renderizaba la barra lateral sin la sesión autenticada, mientras el cliente (navegador) tenía la cookie de Clerk activa. Unificamos el manejo de estado asíncrono en el ``RootLayout`` para lograr una hidratación síncrona sin colapsos visuales.

H3 ****Configuración de Red y Timeouts:****

Debido a que el LLM genera respuestas en streaming (SSE) y a veces tiene alta latencia de inferencia en hardware local, el ruteo asíncrono requirió un rediseño manual en el archivo ``vercel.json`` para extender la tolerancia de red de larga duración en la capa de Vercel y evitar cortes de conexión abruptos.